

# saviour\_project-Copy2

December 21, 2025

```
[1]: import pandas as pd
import numpy as np

df = pd.read_csv("C:\\Users\\hp\\Desktop\\loan.csv")

print("Initial Data Shape", df.shape)
print("\mfinit % row of the data:")
print(df.head())
print("\column Information")
df.info()
```

Initial Data Shape (5000, 17)

\mfinit % row of the data:

|   | customer_id   | loan_id    | loan_type     | loan_amount | interest_rate | \ |
|---|---------------|------------|---------------|-------------|---------------|---|
| 0 | CUST-00004912 | LN00004170 | Car Loan      | 16795       | 0.051852      |   |
| 1 | CUST-00004194 | LN00002413 | Personal Loan | 1860        | 0.089296      |   |
| 2 | CUST-00003610 | LN00000024 | Personal Loan | 77820       | 0.070470      |   |
| 3 | CUST-00001895 | LN00001742 | Car Loan      | 55886       | 0.062155      |   |
| 4 | CUST-00003782 | LN00003161 | Home Loan     | 7265        | 0.070635      |   |

|   | loan_term | employment_type | income_level | credit_score | gender | \ |
|---|-----------|-----------------|--------------|--------------|--------|---|
| 0 | 15        | Self-employed   | Medium       | 833          | Male   |   |
| 1 | 56        | Full-time       | Medium       | 776          | Female |   |
| 2 | 51        | Full-time       | Low          | 697          | Male   |   |
| 3 | 30        | Full-time       | Low          | 795          | Female |   |
| 4 | 48        | Part-time       | Low          | 519          | Female |   |

|   | marital_status | education_level | application_date | approval_date | \ |
|---|----------------|-----------------|------------------|---------------|---|
| 0 | Single         | Master          | 05/04/2018       | 23/04/2018    |   |
| 1 | Married        | Bachelor        | 30/12/2022       | 31/12/2022    |   |
| 2 | Divorced       | High School     | 15/11/2019       | 18/11/2019    |   |
| 3 | Married        | PhD             | 25/08/2021       | 08/09/2021    |   |
| 4 | Married        | High School     | 02/09/2020       | 07/09/2020    |   |

|   | disbursement_date | due_date   | default_status |
|---|-------------------|------------|----------------|
| 0 | 24/04/2018        | 14/08/2018 | False          |
| 1 | 12/01/2023        | 05/04/2023 | False          |
| 2 | 27/11/2019        | 24/02/2020 | False          |

```

3      11/09/2021  25/02/2022      False
4      11/09/2020  29/12/2020      False
\column Information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customer_id           5000 non-null   object
1   loan_id               5000 non-null   object
2   loan_type             5000 non-null   object
3   loan_amount           5000 non-null   int64
4   interest_rate         5000 non-null   float64
5   loan_term             5000 non-null   int64
6   employment_type       5000 non-null   object
7   income_level          5000 non-null   object
8   credit_score          5000 non-null   int64
9   gender               5000 non-null   object
10  marital_status        5000 non-null   object
11  education_level       5000 non-null   object
12  application_date      5000 non-null   object
13  approval_date         5000 non-null   object
14  disbursement_date     5000 non-null   object
15  due_date              5000 non-null   object
16  default_status        5000 non-null   bool
dtypes: bool(1), float64(1), int64(3), object(12)
memory usage: 630.0+ KB

```

```

[2]: date_cols = ['application_date', 'approval_date', 'disbursement_date',
    ↪ 'due_date']
    for col in date_cols:
        df[col] = pd.to_datetime(df[col], format='%d/%m/%Y')

[3]: df['days_app_to_approval'] = (df['approval_date'] - df['application_date']).dt.
    ↪ days
    df['days_approval_to_disbursement'] = (df['disbursement_date'] -
    ↪ df['approval_date']).dt.days
    df['loan_tenor_days'] = (df['due_date'] - df['disbursement_date']).dt.days
    print(df[['days_app_to_approval', 'days_approval_to_disbursement',
    ↪ 'loan_tenor_days']].describe())
    print(df[df['days_app_to_approval'] < 0])

```

|       | days_app_to_approval | days_approval_to_disbursement | loan_tenor_days |
|-------|----------------------|-------------------------------|-----------------|
| count | 5000.000000          | 5000.000000                   | 5000.000000     |
| mean  | 14.396000            | 7.035400                      | 104.614200      |
| std   | 8.559149             | 4.302726                      | 43.062673       |
| min   | 0.000000             | 0.000000                      | 30.000000       |
| 25%   | 7.000000             | 3.000000                      | 67.000000       |

|     |           |           |            |
|-----|-----------|-----------|------------|
| 50% | 14.000000 | 7.000000  | 106.000000 |
| 75% | 22.000000 | 11.000000 | 142.000000 |
| max | 29.000000 | 14.000000 | 179.000000 |

Empty DataFrame

Columns: [customer\_id, loan\_id, loan\_type, loan\_amount, interest\_rate, loan\_term, employment\_type, income\_level, credit\_score, gender, marital\_status, education\_level, application\_date, approval\_date, disbursement\_date, due\_date, default\_status, days\_app\_to\_approval, days\_approval\_to\_disbursement, loan\_tenor\_days]

Index: []

```
[4]: df['default_status'] = df['default_status'].astype(int)
print(df['default_status'].unique())
print(df['default_status'].dtype)
```

[0 1]

int64

```
[5]: cols_to_drop = ['application_date', 'approval_date', 'disbursement_date', 'due_date',
                    'customer_id', 'loan_id']
df = df.drop(columns=cols_to_drop)

print("\nEntire dataset:")
print(df)

print("\nColumn Information:")
df.info()
```

Entire dataset:

|      | loan_type     | loan_amount | interest_rate | loan_term | employment_type | \ |
|------|---------------|-------------|---------------|-----------|-----------------|---|
| 0    | Car Loan      | 16795       | 0.051852      | 15        | Self-employed   |   |
| 1    | Personal Loan | 1860        | 0.089296      | 56        | Full-time       |   |
| 2    | Personal Loan | 77820       | 0.070470      | 51        | Full-time       |   |
| 3    | Car Loan      | 55886       | 0.062155      | 30        | Full-time       |   |
| 4    | Home Loan     | 7265        | 0.070635      | 48        | Part-time       |   |
| ...  | ...           | ...         | ...           | ...       | ...             |   |
| 4995 | Car Loan      | 37945       | 0.070087      | 57        | Self-employed   |   |
| 4996 | Personal Loan | 48937       | 0.056405      | 50        | Part-time       |   |
| 4997 | Home Loan     | 7476        | 0.064212      | 58        | Full-time       |   |
| 4998 | Car Loan      | 52756       | 0.094914      | 12        | Self-employed   |   |
| 4999 | Personal Loan | 91101       | 0.083821      | 52        | Self-employed   |   |

|   | income_level | credit_score | gender | marital_status | education_level | \ |
|---|--------------|--------------|--------|----------------|-----------------|---|
| 0 | Medium       | 833          | Male   | Single         | Master          |   |
| 1 | Medium       | 776          | Female | Married        | Bachelor        |   |
| 2 | Low          | 697          | Male   | Divorced       | High School     |   |
| 3 | Low          | 795          | Female | Married        | PhD             |   |

|      |        |     |        |         |             |
|------|--------|-----|--------|---------|-------------|
| 4    | Low    | 519 | Female | Married | High School |
| ...  | ...    | ... | ...    | ...     | ...         |
| 4995 | High   | 511 | Male   | Married | PhD         |
| 4996 | Medium | 502 | Male   | Single  | PhD         |
| 4997 | High   | 452 | Female | Single  | High School |
| 4998 | Medium | 728 | Male   | Married | PhD         |
| 4999 | Low    | 586 | Male   | Single  | Master      |

|      | default_status | days_app_to_approval | days_approval_to_disbursement | \   |
|------|----------------|----------------------|-------------------------------|-----|
| 0    | 0              | 18                   |                               | 1   |
| 1    | 0              | 1                    |                               | 12  |
| 2    | 0              | 3                    |                               | 9   |
| 3    | 0              | 14                   |                               | 3   |
| 4    | 0              | 5                    |                               | 4   |
| ...  | ...            | ...                  | ...                           | ... |
| 4995 | 0              | 19                   |                               | 3   |
| 4996 | 0              | 5                    |                               | 10  |
| 4997 | 1              | 15                   |                               | 8   |
| 4998 | 0              | 16                   |                               | 14  |
| 4999 | 0              | 1                    |                               | 3   |

|      | loan_tenor_days |
|------|-----------------|
| 0    | 112             |
| 1    | 83              |
| 2    | 89              |
| 3    | 167             |
| 4    | 109             |
| ...  | ...             |
| 4995 | 119             |
| 4996 | 177             |
| 4997 | 153             |
| 4998 | 154             |
| 4999 | 140             |

[5000 rows x 14 columns]

Column Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 5000 entries, 0 to 4999

Data columns (total 14 columns):

| #   | Column          | Non-Null Count | Dtype   |
|-----|-----------------|----------------|---------|
| --- | -----           | -----          | -----   |
| 0   | loan_type       | 5000 non-null  | object  |
| 1   | loan_amount     | 5000 non-null  | int64   |
| 2   | interest_rate   | 5000 non-null  | float64 |
| 3   | loan_term       | 5000 non-null  | int64   |
| 4   | employment_type | 5000 non-null  | object  |
| 5   | income_level    | 5000 non-null  | object  |

```

6   credit_score          5000 non-null   int64
7   gender                5000 non-null   object
8   marital_status        5000 non-null   object
9   education_level       5000 non-null   object
10  default_status        5000 non-null   int64
11  days_app_to_approval  5000 non-null   int64
12  days_approval_to_disbursement 5000 non-null   int64
13  loan_tenor_days       5000 non-null   int64
dtypes: float64(1), int64(7), object(6)
memory usage: 547.0+ KB

```

```
[6]: df.to_csv("cleaned_dataset.csv", index=False)
```

```

[7]: df = pd.read_csv("cleaned_dataset.csv")
print("\nEntire dataset:")
print(df)

print("\nColumn Information:")
df.info()

```

Entire dataset:

|      | loan_type     | loan_amount | interest_rate | loan_term | employment_type \ |
|------|---------------|-------------|---------------|-----------|-------------------|
| 0    | Car Loan      | 16795       | 0.051852      | 15        | Self-employed     |
| 1    | Personal Loan | 1860        | 0.089296      | 56        | Full-time         |
| 2    | Personal Loan | 77820       | 0.070470      | 51        | Full-time         |
| 3    | Car Loan      | 55886       | 0.062155      | 30        | Full-time         |
| 4    | Home Loan     | 7265        | 0.070635      | 48        | Part-time         |
| ...  | ...           | ...         | ...           | ...       | ...               |
| 4995 | Car Loan      | 37945       | 0.070087      | 57        | Self-employed     |
| 4996 | Personal Loan | 48937       | 0.056405      | 50        | Part-time         |
| 4997 | Home Loan     | 7476        | 0.064212      | 58        | Full-time         |
| 4998 | Car Loan      | 52756       | 0.094914      | 12        | Self-employed     |
| 4999 | Personal Loan | 91101       | 0.083821      | 52        | Self-employed     |

|      | income_level | credit_score | gender | marital_status | education_level \ |
|------|--------------|--------------|--------|----------------|-------------------|
| 0    | Medium       | 833          | Male   | Single         | Master            |
| 1    | Medium       | 776          | Female | Married        | Bachelor          |
| 2    | Low          | 697          | Male   | Divorced       | High School       |
| 3    | Low          | 795          | Female | Married        | PhD               |
| 4    | Low          | 519          | Female | Married        | High School       |
| ...  | ...          | ...          | ...    | ...            | ...               |
| 4995 | High         | 511          | Male   | Married        | PhD               |
| 4996 | Medium       | 502          | Male   | Single         | PhD               |
| 4997 | High         | 452          | Female | Single         | High School       |
| 4998 | Medium       | 728          | Male   | Married        | PhD               |
| 4999 | Low          | 586          | Male   | Single         | Master            |

|      | default_status | days_app_to_approval | days_approval_to_disbursement | \  |
|------|----------------|----------------------|-------------------------------|----|
| 0    | 0              | 18                   |                               | 1  |
| 1    | 0              | 1                    |                               | 12 |
| 2    | 0              | 3                    |                               | 9  |
| 3    | 0              | 14                   |                               | 3  |
| 4    | 0              | 5                    |                               | 4  |
| ...  | ...            | ...                  | ...                           |    |
| 4995 | 0              | 19                   |                               | 3  |
| 4996 | 0              | 5                    |                               | 10 |
| 4997 | 1              | 15                   |                               | 8  |
| 4998 | 0              | 16                   |                               | 14 |
| 4999 | 0              | 1                    |                               | 3  |

|      | loan_tenor_days |
|------|-----------------|
| 0    | 112             |
| 1    | 83              |
| 2    | 89              |
| 3    | 167             |
| 4    | 109             |
| ...  | ...             |
| 4995 | 119             |
| 4996 | 177             |
| 4997 | 153             |
| 4998 | 154             |
| 4999 | 140             |

[5000 rows x 14 columns]

Column Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 5000 entries, 0 to 4999

Data columns (total 14 columns):

| #  | Column                        | Non-Null Count | Dtype   |
|----|-------------------------------|----------------|---------|
| 0  | loan_type                     | 5000 non-null  | object  |
| 1  | loan_amount                   | 5000 non-null  | int64   |
| 2  | interest_rate                 | 5000 non-null  | float64 |
| 3  | loan_term                     | 5000 non-null  | int64   |
| 4  | employment_type               | 5000 non-null  | object  |
| 5  | income_level                  | 5000 non-null  | object  |
| 6  | credit_score                  | 5000 non-null  | int64   |
| 7  | gender                        | 5000 non-null  | object  |
| 8  | marital_status                | 5000 non-null  | object  |
| 9  | education_level               | 5000 non-null  | object  |
| 10 | default_status                | 5000 non-null  | int64   |
| 11 | days_app_to_approval          | 5000 non-null  | int64   |
| 12 | days_approval_to_disbursement | 5000 non-null  | int64   |
| 13 | loan_tenor_days               | 5000 non-null  | int64   |

dtypes: float64(1), int64(7), object(6)  
memory usage: 547.0+ KB

```
[8]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import shap
```

```
[9]: from scipy import stats
from datetime import datetime
from sklearn.model_selection import StratifiedKFold, cross_validate, \
    train_test_split, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve, precision_recall_curve,
    confusion_matrix, ConfusionMatrixDisplay
)
from sklearn.linear_model import LogisticRegression
from sklearn.utils.class_weight import compute_class_weight
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline
from xgboost import XGBClassifier
import joblib
from sklearn.metrics import roc_curve, auc
import xgboost as xgb
```

```
[10]: df = pd.read_csv(r"C:\Users\hp\cleaned_dataset.csv")
print("Load full CSV; shape:", df.shape)
print("\nEntire dataset:")
print(df)
print("\nColumn Information:")
df.info()
```

Load full CSV; shape: (5000, 14)

Entire dataset:

|      | loan_type     | loan_amount | interest_rate | loan_term | employment_type \ |
|------|---------------|-------------|---------------|-----------|-------------------|
| 0    | Car Loan      | 16795       | 0.051852      | 15        | Self-employed     |
| 1    | Personal Loan | 1860        | 0.089296      | 56        | Full-time         |
| 2    | Personal Loan | 77820       | 0.070470      | 51        | Full-time         |
| 3    | Car Loan      | 55886       | 0.062155      | 30        | Full-time         |
| 4    | Home Loan     | 7265        | 0.070635      | 48        | Part-time         |
| ...  | ...           | ...         | ...           | ...       | ...               |
| 4995 | Car Loan      | 37945       | 0.070087      | 57        | Self-employed     |

|      |               |       |          |    |               |
|------|---------------|-------|----------|----|---------------|
| 4996 | Personal Loan | 48937 | 0.056405 | 50 | Part-time     |
| 4997 | Home Loan     | 7476  | 0.064212 | 58 | Full-time     |
| 4998 | Car Loan      | 52756 | 0.094914 | 12 | Self-employed |
| 4999 | Personal Loan | 91101 | 0.083821 | 52 | Self-employed |

|      | income_level | credit_score | gender | marital_status | education_level | \ |
|------|--------------|--------------|--------|----------------|-----------------|---|
| 0    | Medium       | 833          | Male   | Single         | Master          |   |
| 1    | Medium       | 776          | Female | Married        | Bachelor        |   |
| 2    | Low          | 697          | Male   | Divorced       | High School     |   |
| 3    | Low          | 795          | Female | Married        | PhD             |   |
| 4    | Low          | 519          | Female | Married        | High School     |   |
| ...  | ...          | ...          | ...    | ...            | ...             |   |
| 4995 | High         | 511          | Male   | Married        | PhD             |   |
| 4996 | Medium       | 502          | Male   | Single         | PhD             |   |
| 4997 | High         | 452          | Female | Single         | High School     |   |
| 4998 | Medium       | 728          | Male   | Married        | PhD             |   |
| 4999 | Low          | 586          | Male   | Single         | Master          |   |

|      | default_status | days_app_to_approval | days_approval_to_disbursement | \ |
|------|----------------|----------------------|-------------------------------|---|
| 0    | 0              | 18                   | 1                             |   |
| 1    | 0              | 1                    | 12                            |   |
| 2    | 0              | 3                    | 9                             |   |
| 3    | 0              | 14                   | 3                             |   |
| 4    | 0              | 5                    | 4                             |   |
| ...  | ...            | ...                  | ...                           |   |
| 4995 | 0              | 19                   | 3                             |   |
| 4996 | 0              | 5                    | 10                            |   |
| 4997 | 1              | 15                   | 8                             |   |
| 4998 | 0              | 16                   | 14                            |   |
| 4999 | 0              | 1                    | 3                             |   |

|      | loan_tenor_days |
|------|-----------------|
| 0    | 112             |
| 1    | 83              |
| 2    | 89              |
| 3    | 167             |
| 4    | 109             |
| ...  | ...             |
| 4995 | 119             |
| 4996 | 177             |
| 4997 | 153             |
| 4998 | 154             |
| 4999 | 140             |

[5000 rows x 14 columns]

Column Information:

<class 'pandas.core.frame.DataFrame'>



RangeIndex: 5000 entries, 0 to 4999

Data columns (total 14 columns):

| #  | Column                        | Non-Null Count | Dtype   |
|----|-------------------------------|----------------|---------|
| 0  | loan_type                     | 5000 non-null  | object  |
| 1  | loan_amount                   | 5000 non-null  | int64   |
| 2  | interest_rate                 | 5000 non-null  | float64 |
| 3  | loan_term                     | 5000 non-null  | int64   |
| 4  | employment_type               | 5000 non-null  | object  |
| 5  | income_level                  | 5000 non-null  | object  |
| 6  | credit_score                  | 5000 non-null  | int64   |
| 7  | gender                        | 5000 non-null  | object  |
| 8  | marital_status                | 5000 non-null  | object  |
| 9  | education_level               | 5000 non-null  | object  |
| 10 | default_status                | 5000 non-null  | int64   |
| 11 | days_app_to_approval          | 5000 non-null  | int64   |
| 12 | days_approval_to_disbursement | 5000 non-null  | int64   |
| 13 | loan_tenor_days               | 5000 non-null  | int64   |

dtypes: float64(1), int64(7), object(6)

memory usage: 547.0+ KB

```
[11]: TARGET = "default_status"
assert TARGET in df.columns, f"Target column '{TARGET}' not found. Available: {df.columns.tolist()}"

X = df.drop(columns=[TARGET])
y = df[TARGET].astype(int)

cat_cols = X.select_dtypes(include=["object", "category"]).columns.tolist()
num_cols = X.select_dtypes(include=["number", "bool"]).columns.tolist()

print(f"Categorical: {cat_cols}")
print(f"Numeric: {num_cols}")
```

Categorical: ['loan\_type', 'employment\_type', 'income\_level', 'gender', 'marital\_status', 'education\_level']

Numeric: ['loan\_amount', 'interest\_rate', 'loan\_term', 'credit\_score', 'days\_app\_to\_approval', 'days\_approval\_to\_disbursement', 'loan\_tenor\_days']

```
[15]: X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42
)
```

```
[16]: categorical_transformer = Pipeline(steps=[
    ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
])
numeric_transformer = Pipeline(steps=[
    ("scaler", StandardScaler())
])
```

```

])
preprocess = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, num_cols),
        ("cat", categorical_transformer, cat_cols)
    ]
)

```

```

[36]: feature_names = pipe_xgb.named_steps["preprocess"].get_feature_names_out()
importances = pipe_xgb.named_steps["model"].feature_importances_

```

```

[37]: fi = pd.DataFrame({
        "Feature": feature_names,
        "Importance": importances
    }).sort_values(by="Importance", ascending=False)
print(fi.head(10))

```

|    | Feature                            | Importance |
|----|------------------------------------|------------|
| 18 | cat__gender_Male                   | 0.045105   |
| 8  | cat__loan_type_Education Loan      | 0.044373   |
| 2  | num__loan_term                     | 0.042934   |
| 3  | num__credit_score                  | 0.042096   |
| 0  | num__loan_amount                   | 0.041473   |
| 1  | num__interest_rate                 | 0.041403   |
| 7  | cat__loan_type_Car Loan            | 0.040931   |
| 20 | cat__marital_status_Married        | 0.040461   |
| 6  | num__loan_tenor_days               | 0.039361   |
| 5  | num__days_approval_to_disbursement | 0.039279   |

```

[19]: cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

```

```

[20]: classes = np.unique(y_train)
class_weights = compute_class_weight(class_weight="balanced", classes=classes,
    ↪ y=y_train)
cw_dict = {int(cls): w for cls, w in zip(classes, class_weights)}
print("Class weights:", cw_dict)

```

```

Class weights: {0: np.float64(0.6248047485160887), 1:
np.float64(2.5031289111389237)}

```

```

[21]: scoring = {
        "accuracy": "accuracy",
        "precision": "precision",
        "recall": "recall",
        "f1": "f1",
        "roc_auc": "roc_auc"
    }

```

```
[22]: pipe_lr = ImbPipeline(steps=[
    ("preprocess", preprocess),
    ("smote", SMOTE(random_state=42, k_neighbors=5)),
    ("model", LogisticRegression(max_iter=3000, class_weight="balanced",
    ↪ solver="lbfgs"))
])

lr_cv = cross_validate(pipe_lr, X, y, cv=cv, scoring=scoring)

pipe_lr.fit(X_train, y_train)
y_pred_lr = pipe_lr.predict(X_test)
y_proba_lr = pipe_lr.predict_proba(X_test)[:, 1]

pipe_xgb = Pipeline([
    ("preprocess", preprocess),
    ("model", xgb.XGBClassifier(n_estimators=200, max_depth=5, learning_rate=0.
    ↪ 1))
])
```

```
[23]: ratio = np.sum(y_train == 0) / np.sum(y_train == 1)

pipe_xgb = Pipeline(steps=[
    ("preprocess", preprocess),
    ("model", XGBClassifier(
        n_estimators=400,
        max_depth=6,
        learning_rate=0.05,
        subsample=0.9,
        colsample_bytree=0.9,
        reg_lambda=1.0,
        random_state=42,
        eval_metric="logloss",
        tree_method="hist",
        scale_pos_weight=ratio
    ))
])

xgb_cv = cross_validate(pipe_xgb, X, y, cv=cv, scoring=scoring)

pipe_xgb.fit(X_train, y_train)
y_pred_xgb = pipe_xgb.predict(X_test)
y_proba_xgb = pipe_xgb.predict_proba(X_test)[:, 1]
```

```
[24]: def get_metrics(y_true, y_pred, y_proba):
    return {
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred, zero_division=0),
```

```

    "Recall": recall_score(y_true, y_pred, zero_division=0),
    "F1": f1_score(y_true, y_pred, zero_division=0),
    "ROC_AUC": roc_auc_score(y_true, y_proba),
}

```

```

res_lr = get_metrics(y_test, y_pred_lr, y_proba_lr)
res_xgb = get_metrics(y_test, y_pred_xgb, y_proba_xgb)

```

```
[25]: pipe_lr.fit(X_train, y_train)
```

```

[25]: Pipeline(steps=[('preprocess',
                        ColumnTransformer(transformers=[('num',
                                                         Pipeline(steps=[('scaler',
                                                         StandardScaler()))]),
                                                         [
('loan_amount',
 'interest_rate', 'loan_term',
 'credit_score',
 'days_app_to_approval',
 'loan_tenor_days']),
('cat',
 Pipeline(steps=[('onehot',
 OneHotEncoder(handle_unknown='ignore',
                 sparse_output=False))])),
('smote', SMOTE(random_state=42)),
('model',
 LogisticRegression(class_weight='balanced', max_iter=3000))])

```

```
[26]: pipe_xgb.fit(X_train, y_train)
```

```

[26]: Pipeline(steps=[('preprocess',
                        ColumnTransformer(transformers=[('num',
                                                         Pipeline(steps=[('scaler',
                                                         StandardScaler()))]),
                                                         [
('loan_amount',
 'interest_rate', 'loan_term',
 'credit_score',
 'days_app_to_approval',
 'loan_tenor_days']),
('cat',
 Pipeline(steps=[('onehot',

```

```

OneHotEncoder(handle_unknown='ignore',
               sparse_output=False))],

               ['loan_...
               feature_types=None, feature_weights=None,
               gamma=None, grow_policy=None,
               importance_type=None,
               interaction_constraints=None, learning_rate=0.05,
               max_bin=None, max_cat_threshold=None,
               max_cat_to_onehot=None, max_delta_step=None,
               max_depth=6, max_leaves=None,
               min_child_weight=None, missing=nan,
               monotone_constraints=None, multi_strategy=None,
               n_estimators=400, n_jobs=None,
               num_parallel_tree=None, ...))]

```

```

[27]: results = {
        "Logistic Regression (SMOTE)": {metric: np.mean(lr_cv[f"test_{metric}"])}
    ↪ for metric in scoring},
        "XGBoost (scale_pos_weight)": {metric: np.mean(xgb_cv[f"test_{metric}"])}
    ↪ for metric in scoring},
    }
results_df = pd.DataFrame(results).T
print("\nCross-Validation Comparative Results:\n", results_df)

```

Cross-Validation Comparative Results:

|                             | accuracy | precision | recall   | f1       | roc_auc  |
|-----------------------------|----------|-----------|----------|----------|----------|
| Logistic Regression (SMOTE) | 0.5252   | 0.208906  | 0.493492 | 0.293420 | 0.519028 |
| XGBoost (scale_pos_weight)  | 0.7288   | 0.206117  | 0.125121 | 0.155451 | 0.510450 |

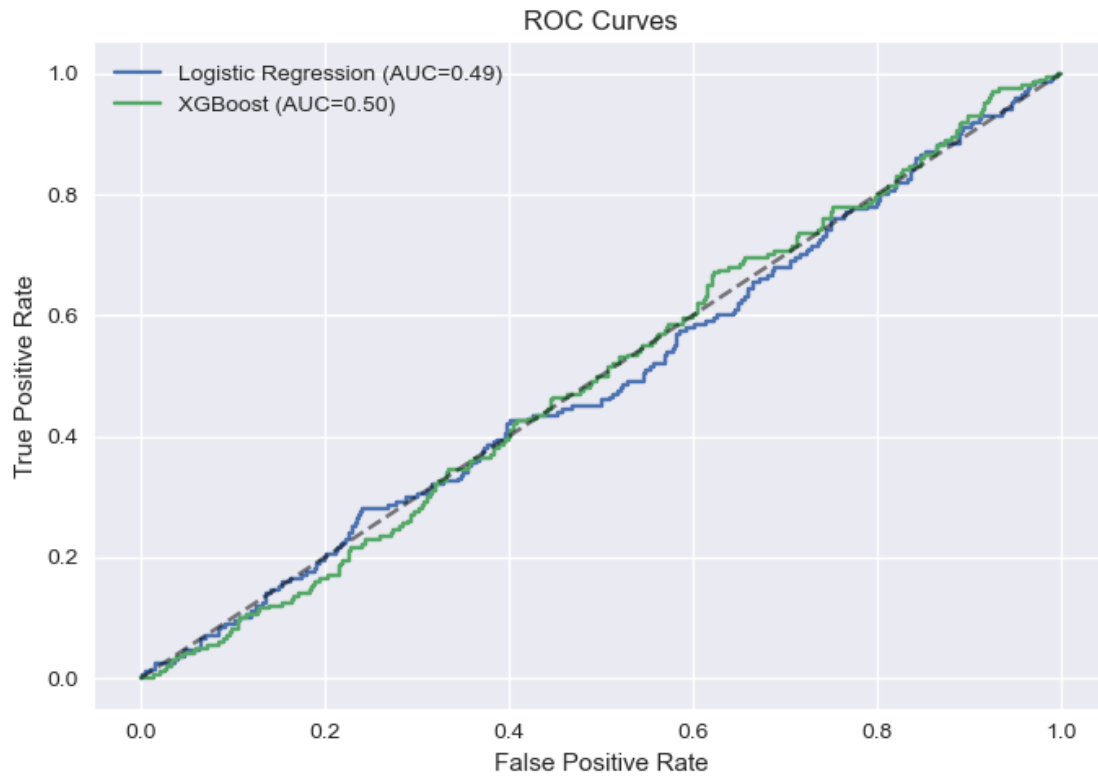
```

[28]: plt.style.use("seaborn-v0_8")

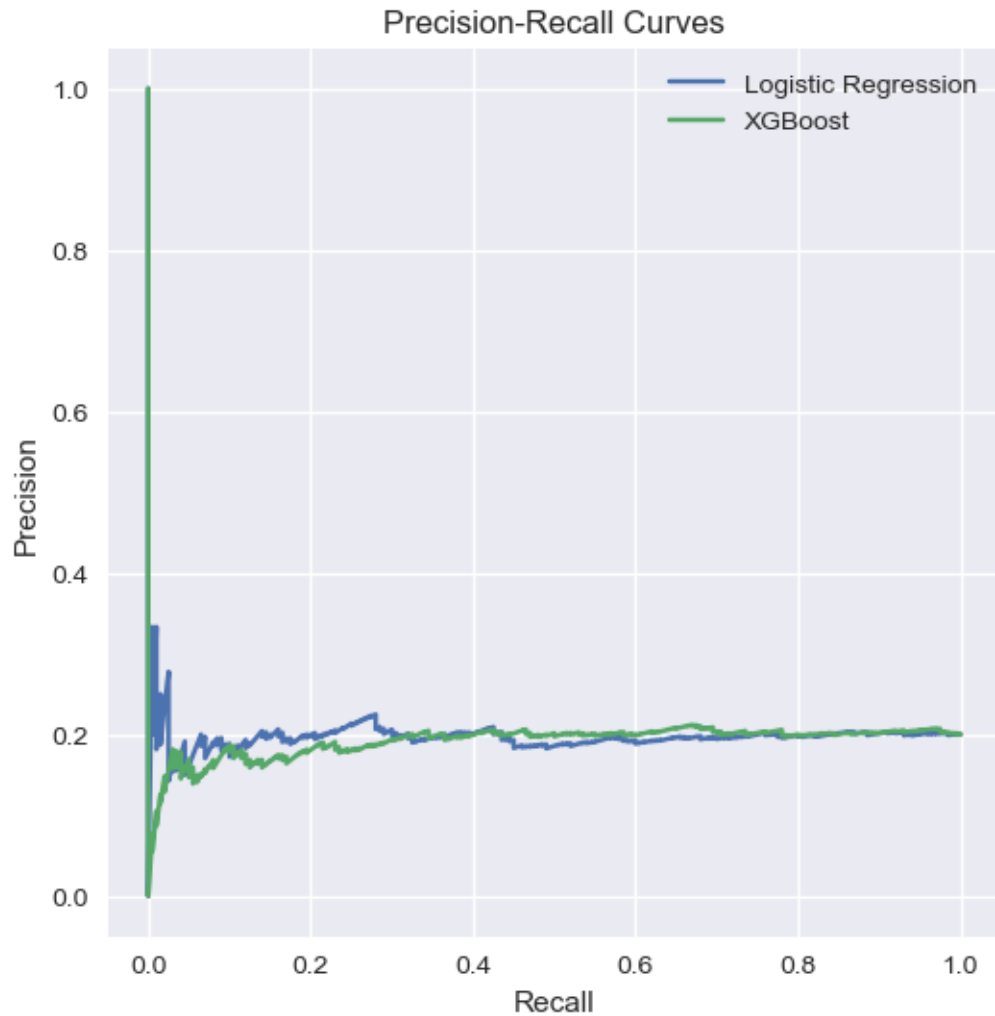
fpr_lr, tpr_lr, _ = roc_curve(y_test, y_proba_lr)
fpr_xgb, tpr_xgb, _ = roc_curve(y_test, y_proba_xgb)

plt.figure(figsize=(7,5))
plt.plot(fpr_lr, tpr_lr, label=f"Logistic Regression (AUC={auc(fpr_lr,tpr_lr):.
    ↪ 2f})")
plt.plot(fpr_xgb, tpr_xgb, label=f"XGBoost (AUC={auc(fpr_xgb,tpr_xgb):.2f})")
plt.plot([0,1],[0,1], 'k--', alpha=0.5)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curves");
plt.legend();
plt.tight_layout()
plt.show()

```

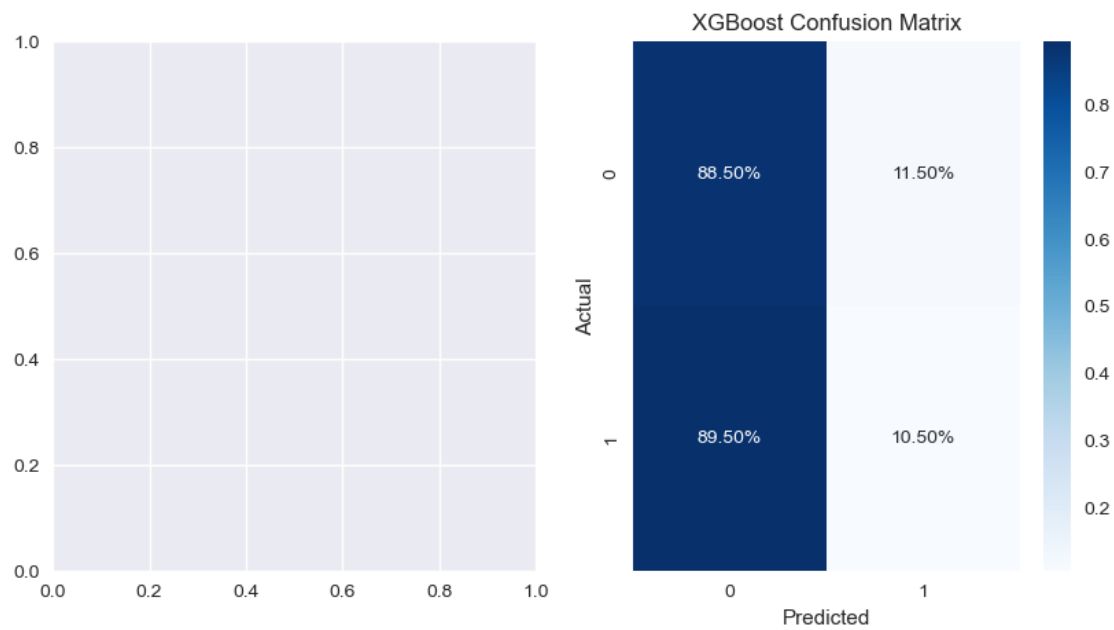
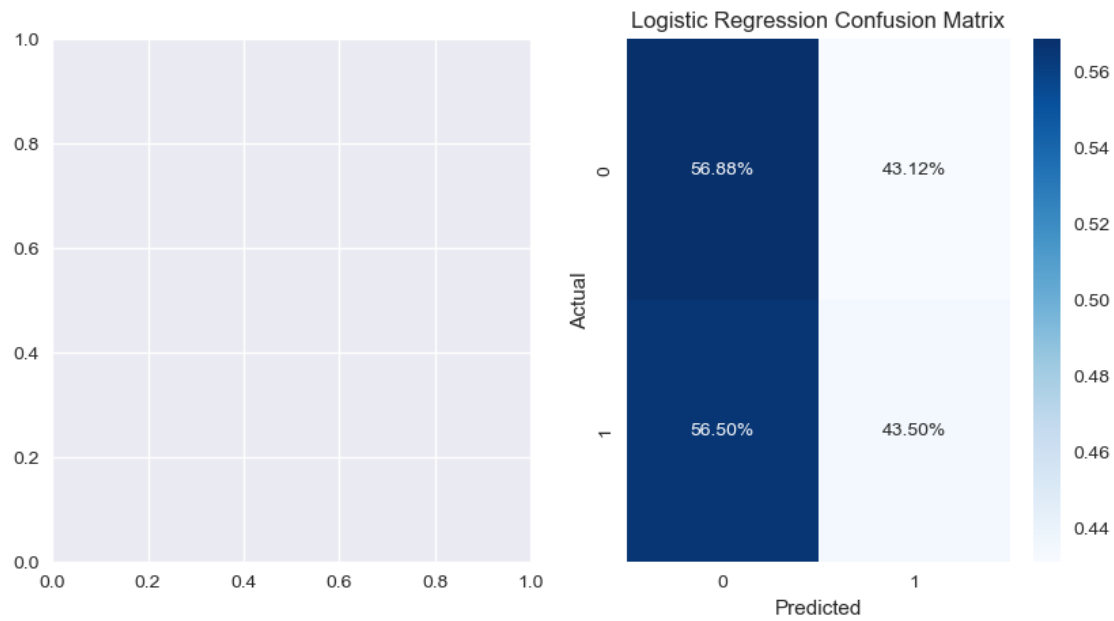


```
[29]: prec_lr, rec_lr, _ = precision_recall_curve(y_test, y_proba_lr)
      prec_xgb, rec_xgb, _ = precision_recall_curve(y_test, y_proba_xgb)
      plt.figure(figsize=(6,6))
      plt.plot(rec_lr, prec_lr, label="Logistic Regression")
      plt.plot(rec_xgb, prec_xgb, label="XGBoost")
      plt.xlabel("Recall"); plt.ylabel("Precision")
      plt.title("Precision-Recall Curves"); plt.legend(); plt.show()
```



```
[30]: def plot_confusion(y_true, y_pred, title):
        cm = confusion_matrix(y_true, y_pred)
        cmn = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
        fig, ax = plt.subplots(1, 2, figsize=(10,5))
        sns.heatmap(cmn, annot=True, fmt=".2%", cmap="Blues", xticklabels=[0,1],
        yticklabels=[0,1])
        plt.ylabel("Actual"); plt.xlabel("Predicted"); plt.title(title)
        plt.show()

        plot_confusion(y_test, y_pred_lr, "Logistic Regression Confusion Matrix")
        plot_confusion(y_test, y_pred_xgb, "XGBoost Confusion Matrix")
```

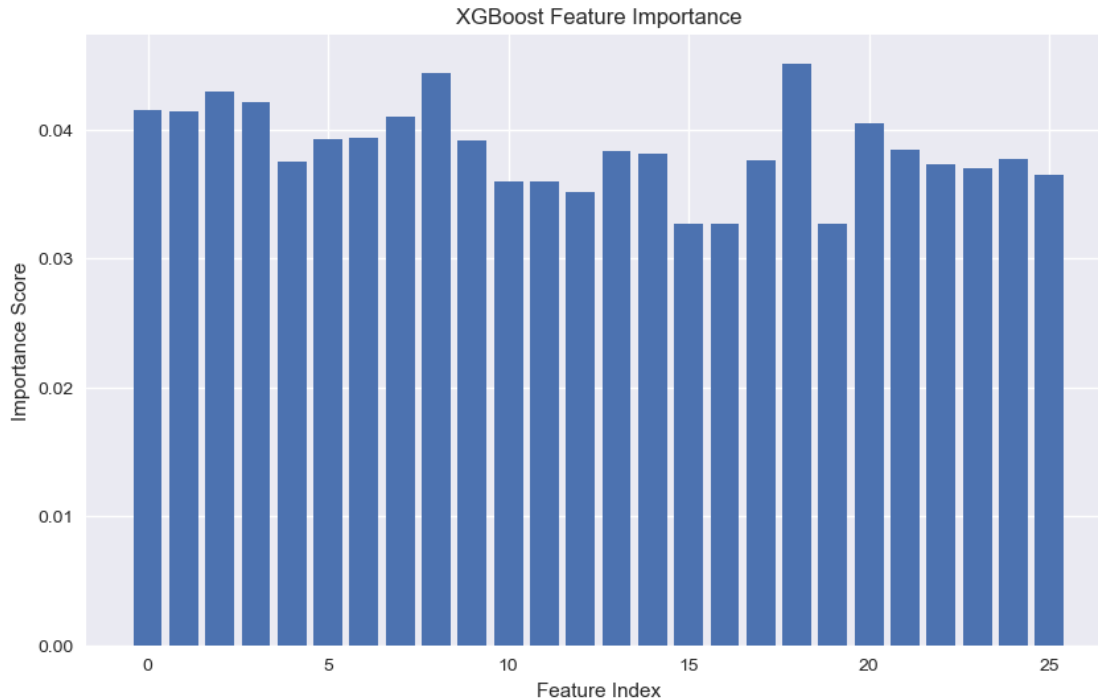


```
[38]: best_xgb = pipe_xgb.named_steps["model"]
plt.figure(figsize=(10,6))
importances = best_xgb.feature_importances_
plt.bar(range(len(importances)), importances)
plt.title("XGBoost Feature Importance")
```



```
plt.xlabel("Feature Index"); plt.ylabel("Importance Score")
plt.show()

print("\nTop features contributing to default risk:\n", fi.head(10))
```

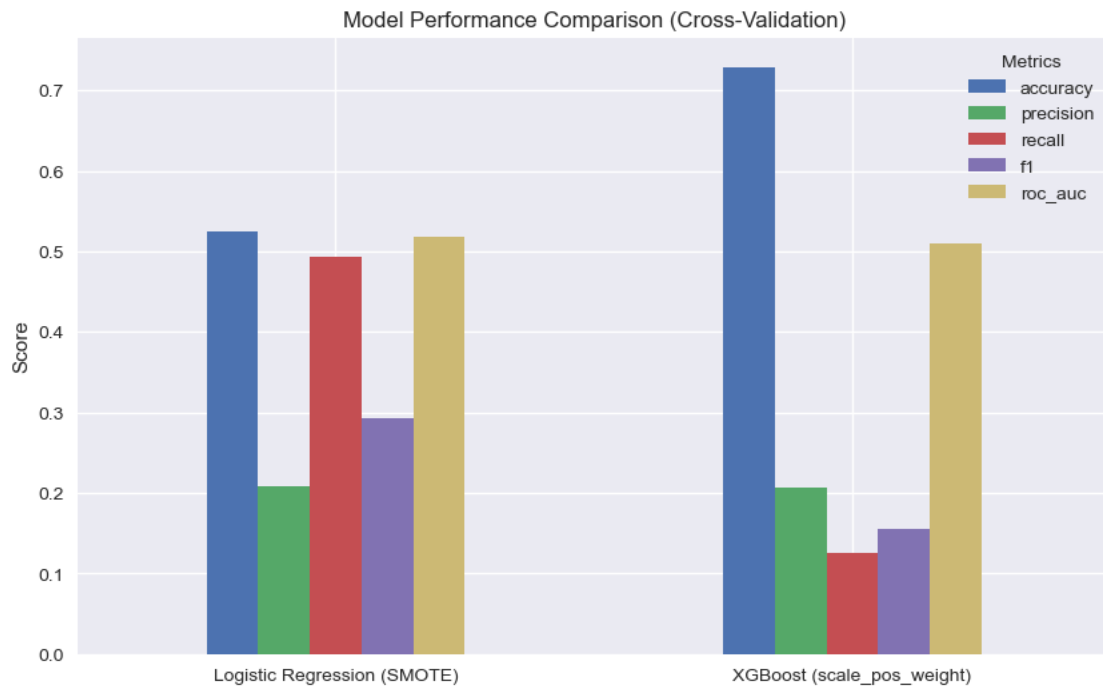


Top features contributing to default risk:

|    | Feature                            | Importance |
|----|------------------------------------|------------|
| 18 | cat__gender_Male                   | 0.045105   |
| 8  | cat__loan_type_Education Loan      | 0.044373   |
| 2  | num__loan_term                     | 0.042934   |
| 3  | num__credit_score                  | 0.042096   |
| 0  | num__loan_amount                   | 0.041473   |
| 1  | num__interest_rate                 | 0.041403   |
| 7  | cat__loan_type_Car Loan            | 0.040931   |
| 20 | cat__marital_status_Married        | 0.040461   |
| 6  | num__loan_tenor_days               | 0.039361   |
| 5  | num__days_approval_to_disbursement | 0.039279   |

```
[38]: results_df.plot(kind="bar", figsize=(10,6))
plt.title("Model Performance Comparison (Cross-Validation)")
plt.ylabel("Score")
plt.xticks(rotation=0)
plt.legend(title="Metrics")
```

```
plt.show()
```



```
[39]: ratio = np.sum(y_train == 0) / np.sum(y_train == 1)
```

```
# Base XGBoost model
xgb = XGBClassifier(
    eval_metric="logloss",
    tree_method="hist",
    random_state=42,
    scale_pos_weight=ratio
)
```

```
[40]: param_grid = {
    "n_estimators": [200, 400, 600],
    "max_depth": [4, 6, 8],
    "learning_rate": [0.01, 0.05, 0.1],
    "subsample": [0.8, 0.9, 1.0],
    "colsample_bytree": [0.8, 0.9, 1.0],
    "reg_lambda": [0.5, 1.0, 2.0]
}
```

```
# Pipeline with preprocessing
pipe_xgb = Pipeline(steps=[
    ("preprocess", preprocess),

```

```
    ("model", xgb)
])
```

```
[41]: cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
      grid_search = GridSearchCV(
          estimator=pipe_xgb,
          param_grid={"model__" + k: v for k, v in param_grid.items()},
          scoring="roc_auc", # optimize for ROC-AUC
          cv=cv,
          n_jobs=-1,
          verbose=2
      )
```

```
[ ]: grid_search.fit(X_train, y_train)
```

Fitting 5 folds for each of 729 candidates, totalling 3645 fits

```
[ ]: print("Best Parameters:", grid_search.best_params_)
```

```
[ ]: best_model = grid_search.best_estimator_

      y_pred = best_model.predict(X_test)
      y_proba = best_model.predict_proba(X_test)[:, 1]
```

```
[ ]: print("\nEvaluation of Tuned XGBoost:")

      print("Accuracy:", accuracy_score(y_test, y_pred))
      print("Precision:", precision_score(y_test, y_pred, zero_division=0))
      print("Recall:", recall_score(y_test, y_pred, zero_division=0))
      print("F1 Score:", f1_score(y_test, y_pred, zero_division=0))
      print("ROC-AUC:", roc_auc_score(y_test, y_proba))
```

```
[ ]:
```